

Proyecto de Investigación — Lenguajes de Interfaz 2026 “A”

40 temas selectos y rúbrica de evaluación (5 categorías)

TecNM Campus Tijuana — Ingeniería en Sistemas Computacionales (SCC-1014)

Agosto 2026

Presentación

Cada estudiante desarrollará **una investigación técnica individual** sobre un tema de *Lenguajes de Interfaz y tecnologías relacionadas*, la publicará en el repositorio del curso mediante el flujo **Fork — Pull Request** y documentará el uso de asistentes de IA (LLM) durante el proceso.

Los 40 temas de esta lista fueron **revisados para evitar duplicados** con las investigaciones ya existentes en `entregas/2025/research/` y en `entregas/2026a/research/`. No se admiten temas repetidos ni variantes menores de un tema ya trabajado; el docente asigna o confirma el tema por Google Classroom.

Entrega esperada

Carpeta personal dentro de `entregas/2026a/research/<nombre-del-tema>/` con:

- **README.md** — título, introducción, desarrollo técnico (mínimo 500 palabras), conclusiones y bibliografía en formato **IEEE**.
- **anexo.md** — bitácora de uso de LLM: prompts reales, resultados obtenidos y reflexión crítica (¿ayudó?, ¿hubo sesgos o errores?). Obligatorio si se usó IA.
- (*Opcional*) código, diagramas, esquemas y PDF de papers de referencia.

Reglas del flujo Fork — Pull Request

- No cambiar la ruta indicada ni renombrar **README.md**.
- No modificar archivos ajenos ni el **README.md** de otras carpetas.
- Un Pull Request por estudiante, con commits claros y descriptivos.
- El PR que altere la estructura del repositorio se **rechaza** sin calificación.

Los 40 temas (2026 “A”)

Programación en ensamblador ARM64 y bajo nivel

1. Modos de direccionamiento en ARMv8-A: acceso pre y post-indexado y su uso eficiente
2. Instrucciones condicionales y predicación en AArch64 (CSEL, CSET) frente a saltos
3. Manejo de cadenas y buffers en ensamblador ARM64 sin biblioteca estándar
4. Aritmética de punto flotante con el conjunto FP/SIMD de AArch64
5. Vectorización con NEON: procesamiento de datos en paralelo
6. Alineación de memoria y penalizaciones por acceso no alineado en ARM
7. Uso de la pila y marcos de activación (*frame pointer*) en AArch64
8. Convención de llamada AAPCS64: paso de parámetros y valores de retorno
9. Ensamblador en línea de GCC: restricciones, *clobbers* y buenas prácticas
10. Programación híbrida C y ASM: enlazado, nombres de símbolos y visibilidad

Interrupciones, llamadas al sistema y sistema operativo

11. Llamadas al sistema de Linux en AArch64 mediante `svc` y la tabla de syscalls
12. Vector de excepciones y niveles de privilegio (EL0–EL3) en ARMv8-A
13. Manejo de interrupciones con el GIC (*Generic Interrupt Controller*)
14. Rutinas de servicio de interrupción en Cortex-M y la tabla NVIC
15. Cambio de contexto a bajo nivel: guardado de registros en un planificador mínimo

Cadena de herramientas y formato binario

16. Formato ELF y secciones (`.text`, `.data`, `.bss`) en ejecutables ARM
17. *Linker scripts* para microcontroladores ARM: mapa de memoria
18. Código de arranque (*startup*) y vector de reset en Cortex-M
19. Relocalización y código independiente de posición (PIC/PIE) en ARM
20. Depuración con GDB y GEF: *breakpoints*, *watchpoints* e inspección de registros
21. Desensamblado y análisis de binarios ARM con `objdump` y Ghidra
22. Macros y directivas del ensamblador GNU `as` para código reutilizable

Programación de dispositivos e interfaces

23. Acceso a periféricos por memoria mapeada (MMIO) y el calificador `volatile`
24. Protocolo 1-Wire: implementación por *bit-banging* en ensamblador
25. Interfaz de teclado matricial y antirrebote (*debounce*) por software
26. Control de displays OLED por I2C (SSD1306) desde registros directos
27. Comunicación USB CDC (puerto serie virtual) en microcontroladores ARM
28. DMA en microcontroladores ARM: transferencias de datos sin intervención de la CPU
29. Generación de PWM y control de servomotores desde el temporizador
30. Lectura de ADC y estrategias de muestreo en sistemas embebidos ARM

RISC-V como reto

31. Conjunto base RV64I: registros y formatos de instrucción
32. Extensión vectorial de RISC-V (RVV) frente a NEON de ARM
33. Llamadas al sistema y ABI en RISC-V Linux
34. Portar una rutina en ensamblador de ARM a RISC-V: guía práctica

Interfaz hombre-máquina, tiempo real y proyecto

35. Diseño de una interfaz máquina-máquina con un protocolo serie propio

- 36. Máquinas de estado finito para interfaces de usuario en firmware
- 37. Medición y optimización de latencia en interfaces de tiempo real
- 38. Perfilado de consumo energético: rutinas en ensamblador frente a C

Seguridad y calidad del código

- 39. Desbordamiento de pila y mitigaciones (*stack canaries*) a nivel ARM
- 40. Pruebas unitarias de código en ensamblador con Unity / Ceedling

Rúbrica de evaluación — 5 categorías (100 puntos)

#	Categoría	Pts	Qué se evalúa
1	Rigor técnico y profundidad	30	Comprensión correcta y profunda del tema; exactitud de los conceptos; fuentes actualizadas y pertinentes; desarrollo técnico de al menos 500 palabras con datos, comparativas o ejemplos verificables.
2	Estructura y claridad del README.md	20	Uso correcto de Markdown; organización lógica (introducción, desarrollo, conclusiones); redacción profesional y ortografía sin faltas graves.
3	Originalidad y análisis crítico	20	Síntesis y redacción propias; el texto no es una copia de la salida de un LLM; hay interpretación, comparación y postura argumentada del estudiante.
4	Uso del repositorio y flujo Fork — Pull Request	15	Ruta y nombre de carpeta correctos; README.md sin renombrar; no se tocan archivos ajenos; commits claros; un solo PR bien descrito.
5	Bitácora de IA (anexo.md) y bibliografía IEEE	15	anexo.md con prompts reales, resultados y reflexión honesta sobre el uso de IA; bibliografía con fuentes confiables (IEEE, libros, papers, sitios oficiales) y formato IEEE correcto.

Escala de desempeño por categoría

Nivel	Porcentaje de la categoría	Descripción
Excelente	90–100 %	Cumple todos los criterios con evidencia sólida y sin observaciones.
Satisfactorio	75–89 %	Cumple lo esencial con observaciones menores.
Suficiente	60–74 %	Cumple parcialmente; faltan elementos o hay imprecisiones.
Insuficiente	0–59 %	No cumple el criterio mínimo o hay copia sin análisis.

Penalizaciones

- **Tema duplicado** (ya existe en 2025/research/ o 2026a/research/): se devuelve el PR sin calificar hasta reasignar tema.
- **Alteración de la estructura del repositorio** o de archivos ajenos: PR rechazado (categoría 4 en 0).

- **Uso de IA no declarado** detectado: categorías 3 y 5 en 0 y reporte de integridad académica según `AI_GUIDANCE.md`.
- **Entrega tardía:** según `POLICIES.md`.

Documento generado para el curso Lenguajes de Interfaz (SCC-1014), semestre 2026 “A”. Referencias del curso: `SYLLABUS.md`, `GRADING.md`, `AI_GUIDANCE.md`.